

Securing the RIC/RTI Communication

E2 Interface Security Design & Implementation Plan

Course:	Secure Communication Design & Implementation
Interface:	E2 — Near-RT RIC ↔ xApp / External CTI Feed
Security focus:	mTLS, ECDSA, AES-256-GCM, HMAC-SHA256, Forward Secrecy
Evaluation:	Week of 18 May 2026

TABLE OF CONTENTS

1. Project Overview & Objectives
.....

2. System Architecture
.....

3. Security Properties & Mechanisms
.....

4. Threat Model
.....

5. Implementation Plan
.....

6. Technology Stack
.....

7. Demo Script
.....

8. Viva Preparation
.....

9. Timeline & Task Allocation
.....

01 1. PROJECT OVERVIEW & OBJECTIVES

This project designs and implements a **cryptographically secure communication layer** for the E2 interface of an Open RAN (O-RAN) Cyber Threat Intelligence (CTI) platform. The system secures communication between a **Near-Real-Time RAN Intelligent Controller (Near-RT RIC)** and untrusted third-party **xApps** or external CTI feeds — two entities that must communicate but cannot inherently trust each other.

This project is directly aligned with the group's Final Year Project on implementing a CTI platform for O-RAN, providing real-world context and motivated design justifications for every security mechanism chosen.

Core Objectives

- Design a secure communication protocol between the Near-RT RIC and untrusted xApps over the E2 interface
- Implement mutual TLS (mTLS) with strict certificate pinning for bidirectional authentication
- Ensure payload confidentiality using AES-256-GCM encryption for STIX threat intelligence objects
- Guarantee message integrity and non-repudiation via ECDSA digital signatures and HMAC-SHA256
- Mitigate real-world O-RAN attack vectors: rogue xApp injection, false telemetry, replay attacks, and MITM
- Demonstrate forward secrecy via TLS 1.3 ephemeral ECDHE key exchange

Why this topic scores highest

- Covers ALL five required security properties: confidentiality, integrity, authentication, non-repudiation, and availability
- Directly aligned with the group's FYP — explicitly rewarded in the project brief
- Real-world O-RAN threat model gives strong, motivated justifications for every design choice
- Live demo is compelling: show rogue xApp rejection, tampered payload detection, and Wireshark ciphertext
- Rich viva depth: each mechanism maps to a named O-RAN attack vector

02 2. SYSTEM ARCHITECTURE

The architecture simulates the O-RAN E2 interface as two independent services: the **Near-RT RIC** (acting as the secure receiver and policy enforcer) and an **xApp** (acting as the untrusted sender of threat telemetry). A third component, the **Certificate Authority (CA)**, issues and manages certificates for both parties.

Component	Role	Trust Level	Interface
Near-RT RIC	CTI platform core — receives, verifies, and processes threat telemetry	Trusted	E2 (server side)
xApp / CTI Feed	Third-party app sending threat intelligence data (STIX format)	Untrusted	E2 (client side)
Certificate Authority	Issues X.509 certificates, manages CRL/OCSP for revocation	Trusted	PKI infrastructure
Attacker (simulated)	Rogue xApp or MITM adversary used in demo to prove mitigations	Adversarial	Simulated

Communication Flow

All communication occurs over a single secured channel. The sequence for each telemetry message is as follows:

1	Handshake — TLS 1.3 mTLS handshake — both parties present X.509 certificates. RIC pins the xApp certificate; any unknown cert aborts the connection.
2	Key exchange — ECDHE (Curve25519) ephemeral key exchange provides session keys with forward secrecy. A new ephemeral key pair is used per session.
3	Payload signing — xApp signs the STIX payload with its ECDSA private key (P-256). Signature is attached to the message header.
4	Payload encryption — AES-256-GCM encrypts the signed payload. The GCM authentication tag provides additional integrity assurance.
5	Nonce + timestamp — A 96-bit random nonce and Unix timestamp are prepended to prevent replay attacks. RIC maintains a nonce cache.
6	Verification — RIC decrypts, verifies the ECDSA signature, checks the timestamp window ($\pm 30s$), and confirms nonce uniqueness before processing.

Each security property required by the course brief is explicitly implemented and justified against O-RAN-specific requirements.

Confidentiality

Mechanism: AES-256-GCM

Symmetric authenticated encryption protects all STIX threat intelligence payloads in transit. GCM mode is chosen over CBC because it provides both encryption and authentication in a single pass, eliminating the need for a separate HMAC. 256-bit key length exceeds NIST recommendations for post-2030 security.

Design justifications:

- Why not AES-128? 256-bit provides a larger security margin appropriate for sensitive CTI data.
- Why GCM? Authenticated encryption prevents padding oracle attacks possible with CBC mode.
- Why not RSA encryption? Asymmetric encryption is too slow for bulk data; RSA is used only for key exchange.

Integrity

Mechanism: HMAC-SHA256 + AES-GCM auth tag

Two-layer integrity: the AES-GCM authentication tag detects any bit-level tampering with the ciphertext. An additional HMAC-SHA256 over the message header (nonce + timestamp + sender ID) detects header manipulation that might bypass GCM verification.

Design justifications:

- HMAC-SHA256 provides 128-bit collision resistance — sufficient for message authentication.
- Double-layer integrity catches attacks that target only the authenticated header.
- SHA-256 is preferred over MD5/SHA-1 which have known collision vulnerabilities.

Authentication

Mechanism: Mutual TLS (mTLS) + certificate pinning

Both the Near-RT RIC and xApp must present valid X.509 certificates signed by the trusted CA. The RIC additionally pins known xApp certificates, rejecting any connection from an xApp whose certificate is not in the pinned set — even if it is CA-signed. This prevents rogue xApps registered with a compromised or external CA.

Design justifications:

- Why mTLS over TLS? Standard TLS only authenticates the server; mTLS authenticates both parties.
- Why certificate pinning? Prevents rogue xApps that obtain fraudulent but CA-valid certificates.
- Certificate pinning is standard practice in O-RAN security specifications (O-RAN.WG11).

Non-repudiation

Mechanism: ECDSA-P256 digital signatures

Every threat telemetry payload is signed by the xApp using its ECDSA private key before encryption. The RIC stores the signature alongside the processed threat record. An xApp cannot later deny having sent a particular threat alert because only it holds the private key corresponding to its CA-issued certificate.

Design justifications:

- Why ECDSA over RSA signatures? ECDSA-P256 provides 128-bit security with a much smaller key size.
- Signatures survive decryption — stored in audit log for forensic non-repudiation.
- Combined with timestamps, signatures prove both identity and time of submission.

Forward Secrecy

Mechanism: TLS 1.3 ECDHE (Curve25519)

TLS 1.3 mandates ephemeral key exchange, meaning a fresh ECDHE key pair is generated for every session. If a long-term private key is later compromised, previously recorded sessions remain unreadable — critical for a threat intelligence platform where historical data is sensitive.

Design justifications:

- TLS 1.3 removes support for static RSA key exchange and weak cipher suites by design.
- Curve25519 is chosen for its resistance to side-channel attacks compared to NIST P-256 for key exchange.
- Forward secrecy limits the blast radius of a compromised xApp to its current session only.

04 4. THREAT MODEL

The threat model follows the STRIDE framework applied to the O-RAN E2 interface. Each threat is mapped to a concrete mitigation implemented in the design.

Threat	STRIDE	Description	Mitigation	Likelihood	Impact
Rogue xApp injection	Spoofing	A malicious actor deploys an unauthorised xApp and attempts to connect to the RIC to inject false threat intelligence, potentially causing the RIC to block legitimate traffic.	mTLS with certificate pinning. The RIC maintains a whitelist of pinned xApp certificates. Any connection without a pinned certificate is rejected at the TLS handshake before any data is exchanged.	High	Critical
False telemetry / CTI poisoning	Tampering	A compromised or malicious xApp sends fabricated threat intelligence (e.g., falsely flagging legitimate UEs as attackers), causing the CTI platform to produce incorrect security policies.	ECDSA signature verification. The RIC verifies each payload's signature against the xApp's pinned public key. Unsigned or incorrectly signed payloads are silently dropped and logged.	High	Critical
Man-in-the-middle (MITM)	Spoofing / Tampering	An adversary intercepts the E2 link between the RIC and an xApp, attempting to read or modify threat telemetry in transit.	TLS 1.3 encryption prevents reading. Certificate pinning prevents the attacker from presenting a forged certificate. Any modification to the ciphertext is detected by the AES-GCM authentication tag.	High	High
Replay attack	Tampering	An adversary captures a valid signed telemetry message and re-sends it later to cause duplicate threat alerts or exhaust RIC resources with stale data.	Nonce + timestamp enforcement. Each message includes a cryptographically random 96-bit nonce and a Unix timestamp. The RIC rejects messages with a timestamp outside a ± 30 second window and maintains a nonce cache to detect duplicates.	Medium	Medium

Key compromise of one xApp	Elevation of privilege	An attacker gains access to an xApp's private key, potentially allowing impersonation of that xApp or decryption of previously recorded sessions.	Certificate revocation via CRL/OCSP removes the compromised xApp from the trusted set. TLS 1.3 forward secrecy ensures that past sessions — recorded before compromise — remain unreadable even with the exposed key.	Medium	High
DoS via telemetry flooding	Denial of service	A compromised xApp floods the RIC with high-volume fake telemetry messages, exhausting processing resources and degrading availability of the CTI platform.	Rate limiting per certificate identity. The RIC enforces a per-xApp message rate limit. Connections exceeding the threshold are throttled and the xApp certificate is flagged for review. mTLS ensures rate limiting is tied to verified identity.	Medium	Medium

05 5. IMPLEMENTATION PLAN

The implementation is divided into four phases. The O-RAN stack does not need to be fully deployed — the Near-RT RIC and xApp are simulated as two networked Python services communicating via a secured local or containerised network.

Phase 1 PKI & Certificate Infrastructure

- Set up a root Certificate Authority using OpenSSL
- Generate X.509 certificates for the Near-RT RIC (server) and each xApp (client)
- Implement certificate revocation list (CRL) endpoint
- Configure certificate pinning logic in the RIC's TLS verification callback
- Test: verify that a self-signed or foreign certificate is rejected

Tools: openssl req, openssl ca, openssl x509, Python ssl module

Phase 2 Secure Transport Layer (mTLS)

- Implement TLS 1.3 server on the Near-RT RIC using Python ssl / Java SSLContext
- Implement TLS 1.3 client on the xApp with mutual authentication
- Enforce cipher suite: TLS_AES_256_GCM_SHA384 only
- Implement certificate pinning check post-handshake
- Test: simulate rogue xApp — confirm handshake rejection and error logging

Tools: Python ssl / asyncio, or Java SSLEngine, or Go crypto/tls

Phase 3 Payload Security (Signatures + Encryption)

- Implement ECDSA-P256 key generation and signing for xApp
- Implement ECDSA signature verification on Near-RT RIC
- Add AES-256-GCM encryption/decryption for STIX payload
- Implement nonce generation (os.urandom(12)) and timestamp embedding
- Implement nonce cache on RIC with TTL-based eviction
- Test: tamper with ciphertext — confirm GCM tag failure; replay a message — confirm rejection

Tools: Python cryptography library (hazmat), or Java JCE, or Go crypto/ecdsa

Phase 4 Integration, Logging & Demo Preparation

- Integrate all components into a runnable two-service system
- Implement audit log: record sender identity, signature, timestamp for each accepted message
- Build demo scripts: legitimate flow, rogue xApp, tampered payload, replay attack
- Run Wireshark capture on the E2 link to demonstrate encrypted traffic
- Prepare threat model diagram and architecture slides for presentation

Tools: Wireshark, Python logging, Docker (optional for isolation)

Component	Technology	Purpose	Justification
Language	Python 3.11+	Primary implementation	Rich crypto libraries, rapid prototyping
TLS library	Python ssl / PyOpenSSL	mTLS transport	Native TLS 1.3 support, certificate callbacks
Crypto library	cryptography (PyCA)	ECDSA, AES-GCM, HMAC	Hazmat layer gives full algorithm control
PKI	OpenSSL CLI	CA, cert generation, CRL	Industry standard, well-documented
Message format	STIX 2.1 (JSON)	Threat intelligence payloads	O-RAN CTI standard format
Network capture	Wireshark	Demo: show encrypted traffic	Proves confidentiality visually
Containerisation	Docker (optional)	Isolate RIC / xApp services	Clean simulation of separate entities
Version control	Git / GitHub	Collaboration	Branch per phase, PR-based review

Sample code skeleton — xApp signing & sending

- `import ssl, json, socket`
- `from cryptography.hazmat.primitives.asymmetric import ec`
- `from cryptography.hazmat.primitives import hashes, serialization`
- `from cryptography.hazmat.primitives.ciphers.aead import AESGCM`
- `# 1. Sign payload with xApp ECDSA private key`
- `signature = xapp_private_key.sign(payload_bytes, ec.ECDSA(hashes.SHA256()))`
- `# 2. Encrypt with AES-256-GCM using session-derived key`
- `nonce = os.urandom(12)`
- `ciphertext = AESGCM(session_key).encrypt(nonce, payload_bytes, None)`
- `# 3. Send over mTLS connection: nonce + signature + ciphertext`

07 7. LIVE DEMONSTRATION SCRIPT

The live demonstration should take 8-10 minutes and prove each security property with a concrete, observable outcome. Run all steps from two terminal windows or tabs.

Step 1 Baseline: legitimate flow

Action: Start the Near-RT RIC server and a legitimate xApp with a valid pinned certificate. Send a STIX threat intelligence payload.

Expected result: RIC accepts the connection, verifies the signature, decrypts successfully, logs the threat record with sender identity. Show the terminal output.

Property proved: *Proves confidentiality, integrity, authentication working together.*

Step 2 Rogue xApp rejection

Action: Start a second xApp with a self-signed certificate (not CA-issued, not pinned). Attempt to connect to the RIC.

Expected result: TLS handshake fails immediately. RIC logs: 'Certificate not in pinned set — connection refused'. No data is exchanged.

Property proved: *Proves mutual authentication and certificate pinning.*

Step 3 Tampered payload detection

Action: Intercept a ciphertext message in transit (use a proxy script) and flip one byte. Forward the modified ciphertext to the RIC.

Expected result: RIC decryption fails with AES-GCM authentication tag mismatch. Message is dropped. Audit log records the tamper attempt with timestamp.

Property proved: *Proves integrity via authenticated encryption.*

Step 4 Replay attack rejection

Action: Capture a valid complete message (nonce + signature + ciphertext). Re-send it 60 seconds later to the RIC.

Expected result: RIC rejects: 'Timestamp outside acceptable window' or 'Nonce already seen'. Duplicate entry is blocked.

Property proved: *Proves replay protection.*

Step 5 Wireshark capture

Action: Run Wireshark on the loopback interface during a legitimate xApp session. Capture the E2 traffic.

Expected result: All captured packets show as TLS Application Data — unreadable ciphertext. No STIX content is visible in the capture.

Property proved: *Proves confidentiality visually — the most compelling moment of the demo.*

Q: Why did you choose mTLS over standard TLS?

A: In O-RAN, the Near-RT RIC cannot inherently trust xApps from third-party vendors. Standard TLS only authenticates the server — the client (xApp) remains unauthenticated. mTLS forces both parties to prove their identity using CA-issued certificates, which is mandatory in the O-RAN security specification (O-RAN.WG11).

Q: Why AES-256-GCM instead of AES-256-CBC?

A: CBC mode provides confidentiality but requires a separate HMAC for integrity, and is vulnerable to padding oracle attacks (POODLE, BEAST). GCM is an authenticated encryption mode — it provides both confidentiality and integrity in a single operation, is parallelisable (faster), and is the default in TLS 1.3.

Q: Why ECDSA instead of RSA for signatures?

A: ECDSA-P256 provides equivalent security to RSA-3072 with a much smaller key size (256 bits vs 3072 bits). Smaller keys mean faster signing and verification — important for a real-time telemetry stream. ECDSA is also the standard in modern PKI and is mandated by TLS 1.3.

Q: How does your design achieve non-repudiation?

A: Each telemetry payload is signed by the xApp's ECDSA private key before encryption. The RIC stores the signature and sender certificate in the audit log. Since only the xApp holds its private key — and the corresponding public key is CA-certified — the xApp cannot later deny having sent a particular threat record. This is the cryptographic definition of non-repudiation.

Q: What happens if an xApp's key is compromised?

A: Two protections apply. First, certificate revocation: the CA issues a CRL or OCSP response marking the certificate as revoked; the RIC checks revocation status on each connection. Second, forward secrecy: TLS 1.3 ECDHE means that even with the compromised long-term key, previously recorded sessions cannot be decrypted because their ephemeral session keys were never transmitted.

Q: How does your design ensure availability?

A: Rate limiting per certificate identity prevents a compromised xApp from flooding the RIC with fake telemetry. Certificate revocation removes a misbehaving xApp from the trusted set. Additionally, the nonce cache has bounded memory usage via TTL-based eviction, preventing memory exhaustion from replay attempts.

Q: Why did you choose the E2 interface rather than A1?

A: The E2 interface carries real-time threat telemetry from xApps — this is the highest-risk attack surface because a malicious xApp injecting false telemetry can directly affect RAN security policy decisions. The A1 interface carries policy from Non-RT RIC to Near-RT RIC, which is a more controlled internal flow. E2 gave us a more compelling threat model and a more dramatic live demonstration.

09 9. TIMELINE & TASK ALLOCATION

The evaluation week starts 18 May. With approximately two weeks of implementation time, the following schedule is recommended for a four-person group.

Week / Days	Task	Owner	Deliverable
Week 1 — Days 1-2	PKI setup: CA, certificates, CRL, pinning logic	Member A	Working CA + cert files
Week 1 — Days 2-4	mTLS transport: RIC server + xApp client	Member B	Secure handshake demo
Week 1 — Days 4-5	ECDSA signing + verification module	Member C	Sign/verify unit tests
Week 1 — Days 5-6	AES-256-GCM encryption + nonce/timestamp	Member D	Encrypted payload tests
Week 2 — Days 1-2	Integration: all modules into unified system	All	End-to-end working system
Week 2 — Days 2-3	Threat demo scripts: rogue xApp, tamper, replay	Member A + B	Demo scripts ready
Week 2 — Days 3-4	Wireshark capture, audit log, documentation	Member C + D	Captures + log output
Week 2 — Days 4-5	Presentation slides + viva rehearsal	All	Final slides, Q&A; drill
Day before eval	Full dry run — all demo steps, time it	All	Smooth 10-min demo

Key success criteria for maximum marks

- Every security property (confidentiality, integrity, authentication, non-repudiation, availability) is implemented AND justified with a specific O-RAN threat
- Live demo shows at least 3 failure cases (rogue xApp, tamper, replay) with clear terminal/log output
- Wireshark capture is prepared in advance as a backup in case live capture has issues
- Each group member can explain any design decision — prepare for individual viva questions
- Algorithm choices are justified comparatively (e.g., why GCM over CBC, why ECDSA over RSA)
- Threat model diagram is shown in the presentation slides with STRIDE categories labelled